



DAWOOD UNIVERSITY OF ENGINEERING & TECHNOLOGY KARACHI

STUDENT NAMES:

- Hafsa Atif (23F-AI-62)
- Laiba Wazir (23F-AI-60)
- Hiba Iqbal (23F-AI-77)

BATCH: 23/F

SUBMITTED TO: SIR HAMZA FAROOQUI

SEMESTER: 6TH

COURSE TITLE:Computer Vision

CREDIT HOURS: 3 (Theory: 2, Practical: 1)

PLANT DISEASE CLASSIFICATION USING DEEP LEARNING

Introduction

This project builds an automated plant disease detection system using leaf images. Two deep learning models — a custom CNN and EfficientNetB0 with transfer learning — were trained on the PlantVillage dataset to classify 38 disease categories. The full pipeline runs on Google Colab using TensorFlow and Keras, covering everything from data loading to final model deployment.

Pipeline Architecture

The system follows a 17-step end-to-end workflow:

Data Acquisition → **Preprocessing & Augmentation** → **Feature Extraction (Sobel / Laplacian)** → **Model Training** → **Fine-Tuning** → **Evaluation & Deployment**

- Dataset downloaded via Kaggle API into Google Colab; directory structure counted (train / validation images and class folders)
- Images resized to 224×224, normalised using EfficientNet's preprocess_input, split 70% train / 15% val / 15% test
- Data augmentation applied: RandomFlip, RandomRotation (0.1), RandomZoom (0.1)
- Sobel filter (ksize=5) and Laplacian filter applied for edge and texture feature visualisation
- EfficientNetB0 base frozen → 5-epoch initial training, then last 20 layers unfrozen for 3-epoch fine-tuning
- Model evaluated via accuracy/loss curves, confusion matrix (Seaborn heatmap), and classification report; saved as .h5

Dataset

Source: New Plant Diseases Dataset (Augmented) — Kaggle (vipooooool). Based on the PlantVillage benchmark with RGB leaf images across 38 classes (26 diseases + 12 healthy plant categories).

Property	Details
Total Training Images	~70,295
Total Validation Images	~17,572
Total Classes	38
Input Resolution	224 × 224 px
Train / Val / Test Split	70% / 15% / 15%

Model Design & Training

CNN from Scratch

- Conv2D (32) → BN → MaxPool , Conv2D (64) → BN → MaxPool , Conv2D (128) → BN → MaxPool → Dropout(0.3)
- Flatten → Dense(256, ReLU) → Dropout(0.5) → Dense(38, Softmax) , ~1.2M parameters
- Optimizer: Adam , Loss: Categorical Crossentropy , Batch: 32 , Image: 128×128 , Epochs: 10

EfficientNetB0 (Transfer Learning)

- Pretrained on ImageNet — base frozen for initial training, last 20 layers unfrozen for fine-tuning
- Head: GlobalAveragePooling2D → Dropout(0.3) → Dense(128, ReLU) → Dense(38, Softmax) , ~4.0M parameters
- Phase 1 — Adam (default lr), 5 epochs , Phase 2 — Adam (lr=1e-5), 3 epochs , Batch: 32 , Image: 224×224

Results & Evaluation

Model	Val Accuracy	Parameters	Training Time	Val Loss
CNN From Scratch	85%	~ 332k	Medium	0.42
EfficientNetB0 — Frozen	95%	~ 4.4M	Fast	0.18
EfficientNetB0 — Fine-Tuned	97%	~ 4.4M	High	0.11

- EfficientNetB0 (fine-tuned) achieved the best performance — 97% validation accuracy with only 8 total training epochs
- Accuracy/loss curves confirmed stable convergence with no overfitting after augmentation and Dropout regularisation
- Confusion matrix (38×38 heatmap) showed strong diagonal dominance; macro-average F1-score ~0.96
- Classes with fewer training samples showed slightly lower recall — expected given class imbalance

Challenges & Limitations

- Colab GPU session timeouts during fine-tuning due to high compute demand of EfficientNetB0
- Custom CNN overfit in early experiments — resolved with Dropout layers and aggressive data augmentation
- Kaggle API credential dependency made dataset loading fragile in shared environments
- Class imbalance across 38 categories affected minority-class precision and recall
- Model cannot generalise to plant species or disease types outside the PlantVillage training set
- No real-world deployment yet — inference performance on field photographs (variable lighting, backgrounds) untested

Future Improvements

- Convert to TensorFlow Lite for real-time mobile / edge deployment for farmers
- Add Grad-CAM visualisation to highlight disease-affected leaf regions and improve interpretability
- Apply MixUp or CutMix augmentation and address class imbalance via weighted sampling
- Build a simple web or mobile app where users can upload a leaf photo and get instant disease diagnosis
- Extend to multi-label classification to detect co-occurring diseases on a single leaf

Run automated hyperparameter search (Keras Tuner / Optuna) to further improve accuracy